



Faculty of Engineering Credit Hours
Communication Systems Engineering Program
Academic Year 2025/2026 – Fall 2025

ECE359

Signal Processing for Multimedia

Handwritten Digit Detection

Group Members

ID	Name
22P0299	Rojeah Adel Shokry Tadross
21P0389	Marian Narouz Ibrahim Angly

Table of Contents

1. Introduction.....	2
Objectives	2
2. Neural Networks Overview	3
2.1 What is a Neural Network?	3
2.2 Structure of the Neural Network Used in This Project.....	3
2.3 Mathematical Model of a Neuron	3
2.4 Activation Functions	4
2.4.1 ReLU.....	4
2.4.2 Softmax	4
2.4.3 Loss Function: Sparse Categorical Cross-Entropy.....	5
2.5 Learning Process	5
3. Dataset and Preprocessing	6
3.1 MNIST dataset	6
3.2 Preprocessing decisions	6
4. Model Architecture	6
4.1 Structure	6
4.2 Layer-by-layer justification.....	6
5. Training	7
5.1 Compile and fit	7
5.2 Choices explained	7
6. Evaluation & Predictions	8
6.1 Basic evaluation	8
6.2 Predictions and argmax.....	8
7. Confusion Matrix and Deeper Diagnostics.....	9
7.1 Why use a confusion matrix?	9
7.2 How to compute and display	9
8. Model Saving and Reuse.....	10
9. Results and Discussion	11
10. Samples	11
11. Conclusion	14

1. Introduction

Handwritten digit recognition is a classic problem in computer vision and machine learning. It requires classifying 28×28 grayscale images into one of ten classes (digits 0–9). MNIST is a widely used benchmark dataset for this task and is ideal for demonstrating end-to-end ML workflows: preprocessing, model building, training, evaluation, and deployment.

Objectives

- Implement a simple, reproducible pipeline that loads MNIST, preprocesses it, and trains a dense neural network.
- Explain each code block clearly and justify design choices.
- Evaluate model performance using accuracy and deeper diagnostics (confusion matrix, classification report).
- Provide saved model and instructions for reuse.

This report documents the final version of our handwritten digit detection project using the MNIST dataset and TensorFlow/Keras. The work covers data loading, preprocessing, model design, training, evaluation, saving/loading models, and analysis of prediction errors using a confusion matrix. The goal is educational clarity and reproducible results.

2. Neural Networks Overview

2.1 What is a Neural Network?

A **neural network** is a machine learning model inspired by the structure of the human brain.

It consists of interconnected processing units called **neurons**, which are organized into **layers**.

In supervised learning, neural networks learn a mapping between:

- **Input data** (images)
- **Target labels** (digits 0–9)

by adjusting internal parameters during training.

2.2 Structure of the Neural Network Used in This Project

The implemented model is a **fully connected feedforward neural network**, also known as a **Multilayer Perceptron (MLP)**.

It consists of:

- **Input layer**: flattened image pixels
- **Two hidden layers**: feature extraction and pattern learning
- **Output layer**: digit classification

Each neuron in one layer is connected to **all neurons in the next layer**.

2.3 Mathematical Model of a Neuron

Linear Combination

The neuron computes a weighted sum of its inputs:

$$z = \sum_{i=1}^n w_i x_i + b$$

Where:

- x_i : input features
- w_i : learned weights
- b : bias term
- z : pre-activation output

2.4 Activation Functions

2.4.1 ReLU

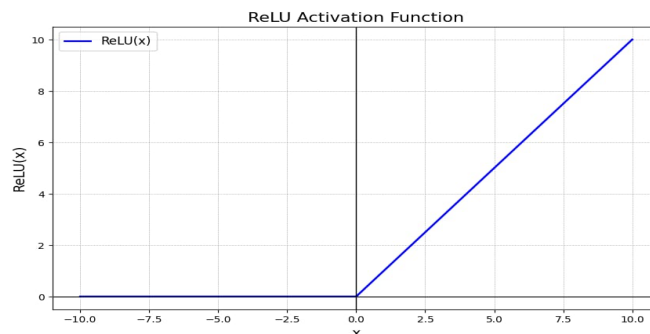
For hidden layers, **ReLU** is used:

$$\text{ReLU}(z) = \max(0, z)$$

This means:

- Negative values are set to zero
- Positive values are preserved

ReLU allows the network to learn complex patterns efficiently and improves convergence speed.



Pros: simple, non-saturating for $x > 0$, sparse activations.

2.4.2 Softmax

The output layer contains **10 neurons**, one for each digit class (0–9).

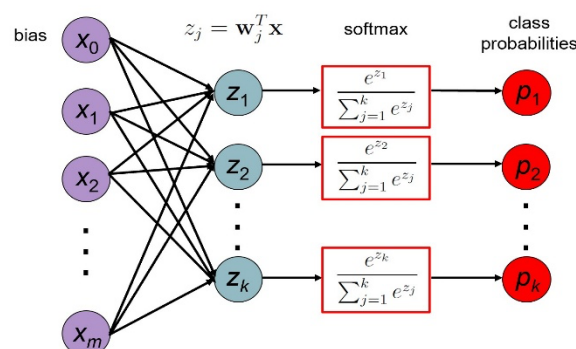
The **Softmax function** converts raw outputs into probabilities:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$$

This ensures:

- All outputs are between 0 and 1
- The sum of probabilities equals 1

The predicted class is the one with the highest probability.



2.4.3 Loss Function: Sparse Categorical Cross-Entropy

To measure prediction error during training, the **sparse categorical cross-entropy** loss function is used:

$$\mathcal{L} = -\log(p_y)$$

Where:

p_y is the predicted probability of the true class

This loss function is suitable because:

The problem is multi-class classification

Labels are integer encoded (0–9)

The goal of training is to **minimize this loss**.

2.5 Learning Process

During training:

The network makes predictions

The loss is computed

Gradients are calculated using backpropagation

Weights are updated using the **Adam optimizer**

This process is repeated for multiple epochs until the model converges.

3. Dataset and Preprocessing

3.1 MNIST dataset

- 60,000 training examples, 10,000 test examples
- Each image: 28×28 grayscale pixels (values 0–255)
- Labels: integers 0–9

3.2 Preprocessing decisions

- **Normalization:** We scale pixel values to the range [0, 1] by dividing by 255.0 (recommended for MNIST). This keeps pixel intensities proportional and works well with ReLU activations.

```
x_train = x_train / 255.0
```

```
x_test = x_test / 255.0
```

- **Rationale:** Simple, predictable, and commonly used. Avoids unintuitive distortions introduced by L2-normalization when applied naively to raw pixel arrays.

4. Model Architecture

4.1 Structure

The model is a feedforward Multi-Layer Perceptron (MLP) using Keras Sequential API:

```
model = tf.keras.models.Sequential()  
model.add(tf.keras.layers.Flatten())  
model.add(tf.keras.layers.Dense(128, activation=  
tf.nn.relu))  
model.add(tf.keras.layers.Dense(128, activation=  
tf.nn.relu))  
model.add(tf.keras.layers.Dense(10, activation='softmax'))
```

4.2 Layer-by-layer justification

- **Flatten:** converts each 28×28 image into a 784-length vector for Dense layers.
- **Dense(128, relu) × 2:** two hidden layers with 128 neurons each.
- **Dense(10, softmax):** output probabilities for the 10 digit classes. Softmax is appropriate for mutually exclusive multi-class classification.

5. Training

5.1 Compile and fit

compilation and training configuration used in the final code:

```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])  
  
model.fit(x_train, y_train,  
          epochs=15,  
          validation_data=(x_test, y_test),  
          batch_size=32)
```

5.2 Choices explained

- **Optimizer – Adam:** robust default with adaptive learning rates; requires minimal tuning.
- **Loss – sparse_categorical_crossentropy:** works directly with integer labels (no one-hot encoding required).

Measures discrepancy between predictions and targets.

(use sparse_categorical_crossentropy when targets are integer labels)

- **Epochs and batch size:** chosen as a balance between training time and performance; these are tunable hyperparameters.

An epoch represents one complete pass of the training data through the neural network. During each epoch, the model processes all training samples and updates its weights to reduce prediction error.

Using multiple epochs allows the model to gradually improve its performance. However, too many epochs can lead to overfitting, where the model memorizes the training data instead of generalizing well to new data.

In this project, the number of epochs was chosen to achieve good accuracy while maintaining stable validation performance.

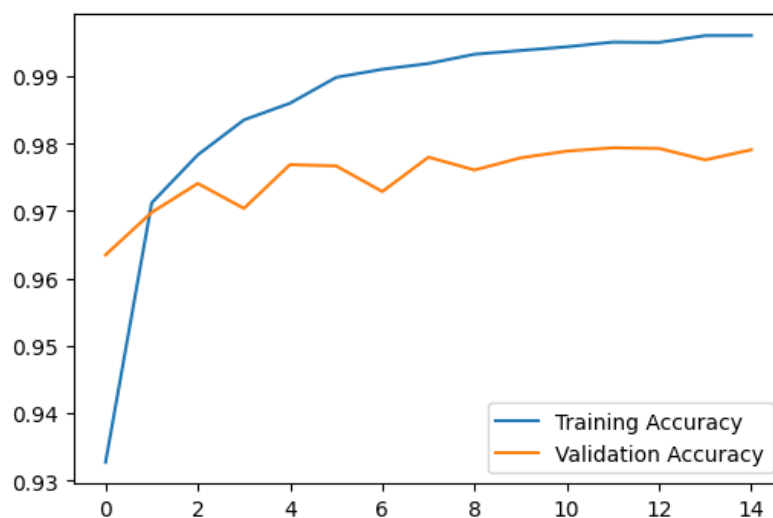
6. Evaluation & Predictions

6.1 Basic evaluation

Evaluate on the held-out test set:

```
val_loss, val_acc = model.evaluate(x_test, y_test)
print(val_loss, val_acc)
```

```
313/313 ————— 1s 2ms/step - accuracy: 0.9769 - loss: 0.1101
0.09704068303108215 0.9790999889373779
Validation Accuracy: 97.91%
```



The training and validation accuracy curves show steady improvement with a small, stable gap between them, indicating effective learning and good generalization without significant overfitting.

6.2 Predictions and argmax

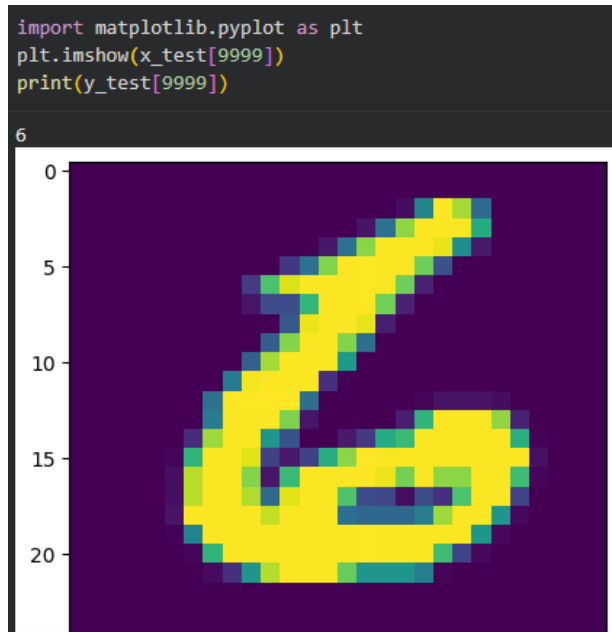
Convert model output probabilities to predicted labels:

```
predictions = new_model.predict(x_test)
print(predictions)
import numpy as np
print(np.argmax(predictions[9999]))
```

```
import numpy as np
print(np.argmax(predictions[9999]))
```

```
6
```

and that is plot for image before preprocessing and prediction:



7. Confusion Matrix and Deeper Diagnostics

7.1 Why use a confusion matrix?

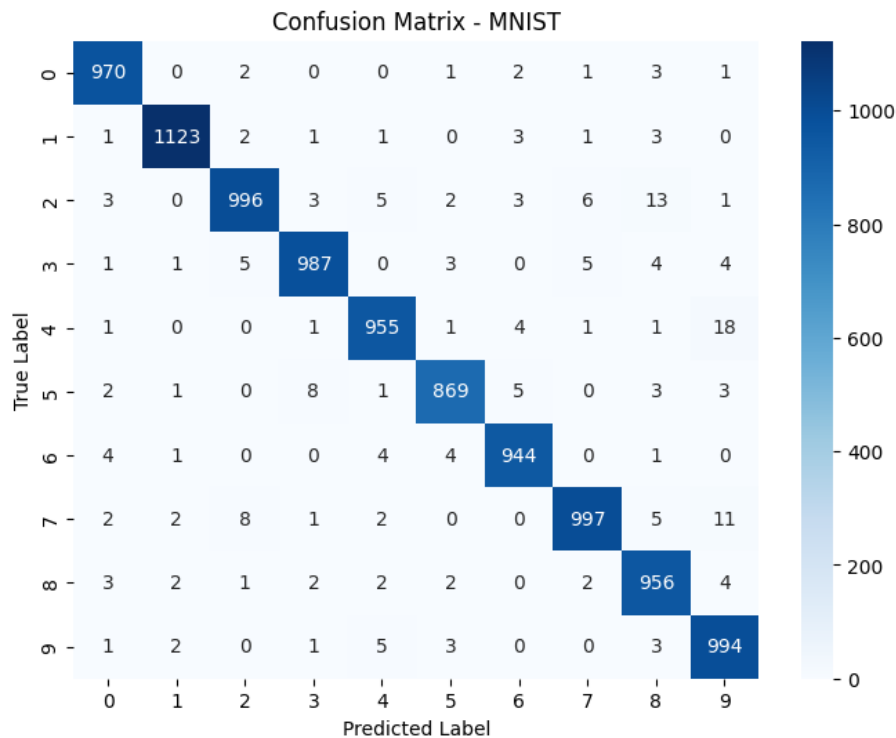
Accuracy summarizes overall correctness but hides which classes are confused. A confusion matrix shows, for each true class, how many samples were predicted as each class. This helps identify systematic mistakes (e.g., 4 $\rightarrow$ 9).

7.2 How to compute and display

```
import numpy as np
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Predict test data
y_pred_prob = model.predict(x_test)
y_pred = np.argmax(y_pred_prob, axis=1)

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
```



The confusion matrix provides a detailed view of the classification performance for each digit.

The diagonal elements represent correctly classified samples, while off-diagonal elements represent misclassifications.

From the matrix, it is clear that most predictions lie along the diagonal, indicating high overall accuracy.

Misclassifications mainly occur between visually similar digits such as **4 and 9**, **7 and 9**, and **2 and 8**, which is expected due to similarities in handwriting styles.

These errors indicate ambiguity in handwritten digit shapes rather than systematic model failure.

Overall, the confusion matrix confirms that the model generalizes well and performs consistently across all digit classes.

8. Model Saving and Reuse

Save the trained model for later use without retraining:

```
model.save('epic_num_reader.keras')
# Load later with
new_model =
tf.keras.models.load_model('epic_num_reader.keras' ,
custom_objects={'softmax': tf.nn.softmax})
```

9. Results and Discussion

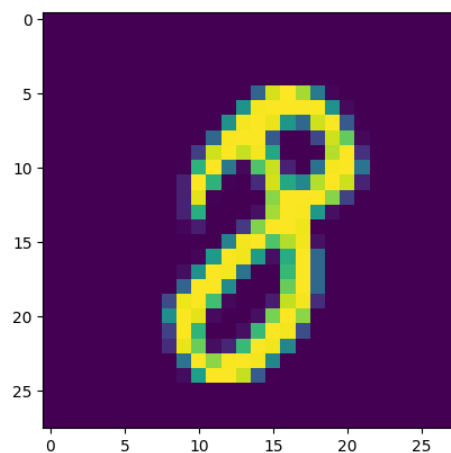
Note: Exact numeric results (test accuracy, confusion matrix values) depend on running the notebook. The code will output these when executed. Here we provide the interpretation framework:

- If accuracy is high ($\approx 97\text{--}99\%$), the model works well for MNIST. Expect slightly lower results for the MLP compared to a CNN.
- Use the confusion matrix to find commonly confused pairs (e.g., 4 & 9, 3 & 5).

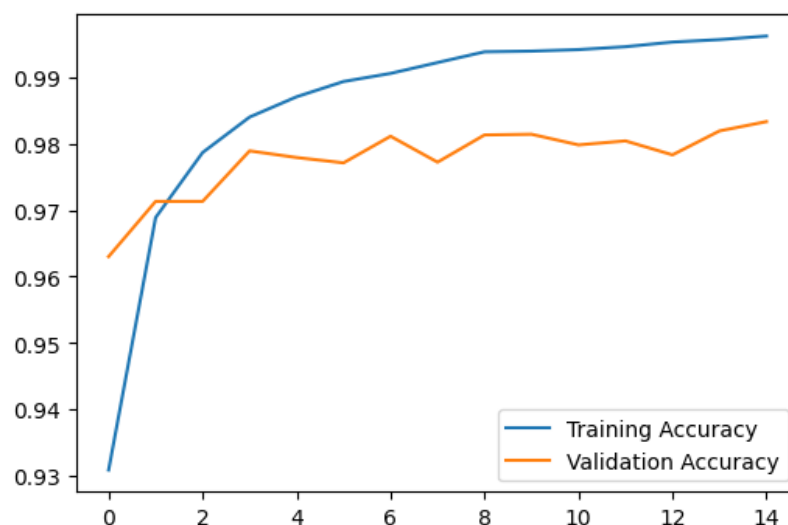
10. Samples

1) sample number 7200

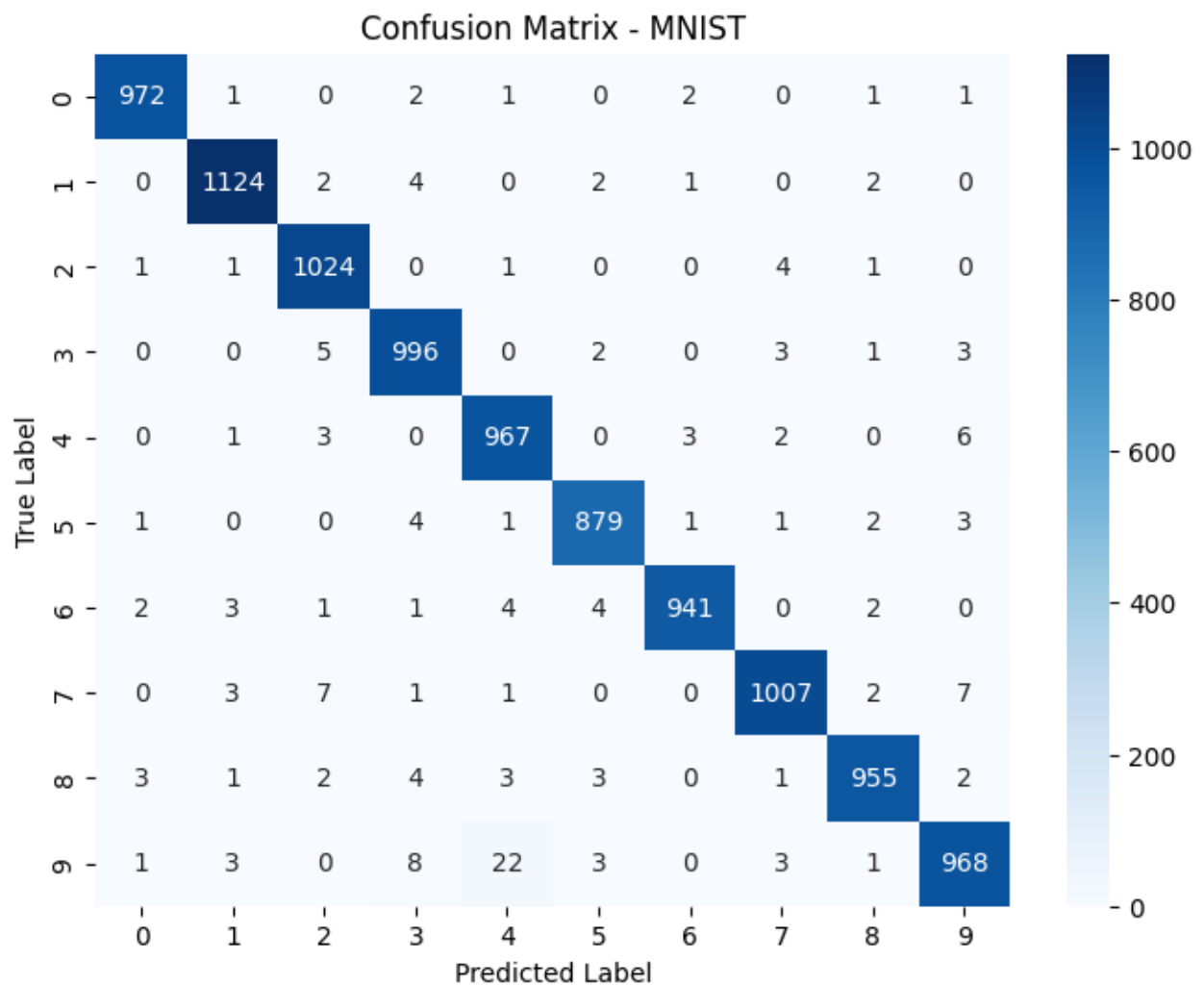
- Before preprocessing:



- Training and accuracy plot



- Confusion matrix



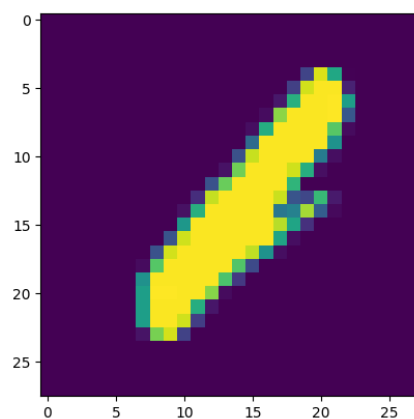
- Prediction number

```
print(np.argmax(predictions[7200]))
```

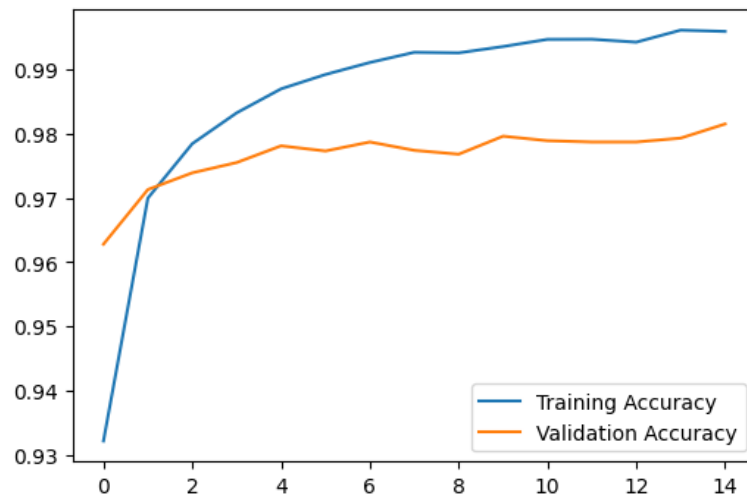
8

2) sample number 1900

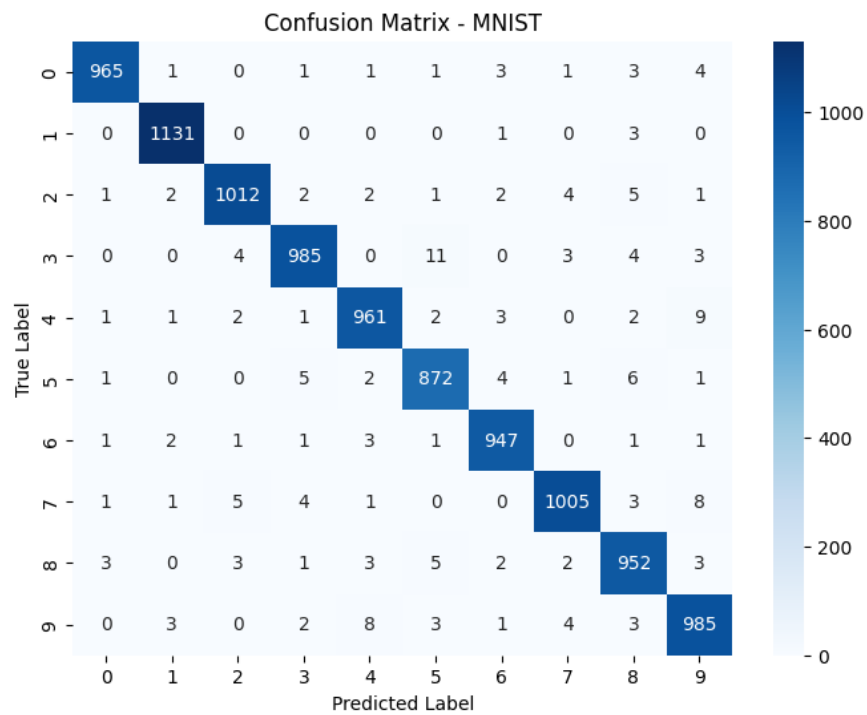
- Before preprocessing:



- Training and accuracy plot



- Confusion matrix



- Prediction number

```
print(np.argmax(predictions[1900]))
```

1

11. Conclusion

In this project, a handwritten digit detection system was implemented using a neural network and the MNIST dataset. The model was trained to classify digit images into classes from 0 to 9 after applying basic data preprocessing and normalization.

The neural network used dense layers with ReLU activation to learn important features from the images, while the Softmax output layer produced probability-based predictions. The model achieved high accuracy on unseen test data, and the training and validation results showed stable learning behavior without significant overfitting.

The confusion matrix analysis confirmed that most digits were correctly classified, with misclassifications mainly occurring between visually similar digits. This behavior is expected in handwritten digit recognition tasks.

Overall, the project successfully demonstrated the complete machine learning pipeline, from data preparation and model training to evaluation and prediction. Future improvements may include using convolutional neural networks to further improve performance.